

Ödev 19: Bir İndirme Servisi

Farkı kendiniz ölçün, sonra fark etmeden kaldırdığınız hız sınırını bulun.

Birinci Bölüm: Ölçün

`DownloadService.java` dosyasını klasik biçimde oluşturun.

Her biri 100ms bloke olan 2.000 simüle indirmeyi üç biçimde çalıştırın:

1. `newVirtualThreadPerTaskExecutor()`
2. `newFixedThreadPool(100)`
3. `newFixedThreadPool(25)`

Üç zamanlamayı da yazdırın.

Sonra iki platform sayısını bakmadan **tahmin edin**. Formül basit ve aynı anda kaç görevin bloke olabileceğinden çıkıyor.

Tahmin ettiğiniz alt sınırları ölçümlerin yanında yazdırın.

Açıklayın

Şunları kapsayan bir yorum:

- Platform sayılarının neden `(görev / havuzBoyutu) × gecikme` olduğu
- Sanal sayının içinde neden `havuzBoyutu` olmadığı
- Asıl nokta.** Platform zamanlamaları tahmin etmek zorunda kaldığınız bir sayıya bağlı. Onu çok düşük tahmin etmenin neye, çok yüksek tahmin etmenin neye mal olduğunu söyleyin.

İkinci Bölüm: Kaldırdığınız Sınır

Alt servis **5 eşzamanlı çağrıya** dayanıyor. Eski kodunuz `newFixedThreadPool(5)` kullanıyordu, dolayısıyla o sınır kazara uygulanıyordu.

Aynı anda uçuşta olan çağrıların **tepe** sayısını kaydeden bir `ConcurrencyTracker` yazın. Sonra 40 çağrıyı üç biçimde çalıştırıp her seferinde tepeyi yazdırın:

1. 5'lik platform havuzu
2. Sanal iş parçacıkları, **başka hiçbir değişiklik olmadan**
3. Sanal iş parçacıkları artı açık bir `Semaphore(5)`

İkincisi 40'a çıkacak. Hiçbir şey başarısız olmayacak, hiçbir şey uyarmayacak.

Sayacınızın tepe değeri kendisi de yarışsız olmalı. Aynı anda tepeyi yükselten iki iş parçacığı bir güncellemeyi kaybetmemeli, ve bunu nasıl yapacağınızı çözmek alıştırmamızın bir parçası.

Açıklayın

Şunları kapsayan bir yorum:

- Eski havuz boyutunun aynı anda hangi iki ayrı kararı verdiği
- İkisi de 5 verse bile `Semaphore` seçeneğinin havuzdan neden daha iyi olduğu
- `release()` **nereye gitmeli, ve bir istisnada atlanırsa ne olur.** Başarısızlık biçimi konusunda ayrıntılı olun

Üçüncü Bölüm: Kapsam Değerleri

Bir `ScopedValue` ile bir istek kimliği bağlayın, sonra onu parametre olarak geçirmeden **iki çerçeve derinde** okuyun.

Kapsamın dışında bağlı olmadığını gösterin.

Sonra kapsamın içinde bir çocuk iş parçacığı başlatın ve bağlamayı görüp görmediğini gösterin.

Açıklayın

Şunlar hakkında bir yorum:

- Bir `ThreadLocal` ile karşılaştırıldığında **yazmak zorunda kalmadığınız** şey
- Bir `ThreadLocal` metodunun istek bittikten sonra hâlâ neyi tutacağı, ve iş parçacığı tekrar kullanıldığında bunun neden önemli olduğu
- Bir çocuk iş parçacığının bağlamayı neden miras almadığı, ve alması için neyin gerektiği

Dördüncü Bölüm: Değişmeyen Şey

Düz bir `int` alanı her biri 100.000 kez artıran dört sanal iş parçacığı çalıştırın.

Beklenen ile gerçekleşeni yazdırın.

Sanal iş parçacıklarının burada neden yardımcı olmadığını, ve on bin iş parçacığı oluşturmak artık kolay olduğuna göre bu hatanın riskinin **arttığını mı azaldığını mı** bir yorumda yazın.

Kabul Kriterleri

- ☐ `java DownloadService.java` ile çalışıyor
- ☐ Üç zamanlama, tahmin edilen alt sınırlarla birlikte yazdırılıyor

- ☐ Açıklama havuz boyutunu yanlış tahmin etmenin her iki yönünü de kapsıyor
- ☐ `ConcurrencyTracker` bir tepe kaydediyor ve kendisi yarışsız
- ☐ Üç tepe değeri 5, 40 ve 5
- ☐ Bir yorum havuz boyutunun birleştirdiği iki kararı isimlendiriyor
- ☐ `release()` bir `finally` içinde, değilse oluşacak sızıntıyı anlatan yorumla
- ☐ Bir kapsam değeri, parametre geçirmeden iki çerçeve derinde okunuyor
- ☐ Kapsam dışında bağlı olmadığı gösteriliyor
- ☐ Çocuk iş parçacığı davranışı gösteriliyor ve açıklanıyor
- ☐ Güvensiz sayacın sanal iş parçacıkları altında artış kaybettiği gösteriliyor

İleri Seviye

`Semaphore` yerine sınırlı bir kuyruk koyun: iş gönderen bir üretici ve onu boşaltan sabit sayıda sanal iş parçacığı tüketicisi.

Sonra ikisini karşılaştıran bir yorum yazın. Hangisi gönderene geri basınç uyguluyor? İş, işlenebileceğinden hızlı geldiğinde her birinde ne oluyor, ve hangi başarısızlığı hata ayıklamayı tercih edersiniz?

İpucu, sayacınızın tepe değeri düşük görünüyorsa

`peak = Math.max(peak, current)` bir oku-değiştir-yaz işlemidir, ki bu Modül 18'in yarışının farklı değişken adlarıyla hâlidir. `AtomicInteger` sınıfında kazanana kadar yeniden deneyen bir metot var; bulun.